EkitiRun

Race platform for the Ekiti Marathon · Team Task Split · Version 1

Who does what

Person	Share	What they build
Lead (expert)	70%	Repo and CI, database schema, phone OTP, bib numbers with signed QR codes, Paystack webhook, PDF pipeline, offline scanner, timing engine, position estimate, live updates, deployment and load test
Frontend 1	10%	Multi-step registration form, OTP screen, bib and QR confirmation page, sponsor tiers, pledge and thank-you pages
Frontend 2	10%	Live leaderboard, runner search and detail page with splits, Leaflet route map with the runner's estimated position
Backend	10%	Sponsor tiers endpoints, sponsor pledge endpoint, public runner search and lookup, invoice and receipt templates, pytest tests

Contents: Overview · Lead (70%) · Frontend 1 (10%) · Frontend 2 (10%) · Backend (10%) · API contract

EkitiRun: Team Task Split (Version 1)

EkitiRun is the platform. The event keeps its own official name (e.g. "Ekiti Marathon 2027"), so EkitiRun can host future editions and other races.

Version 1 scope

Free runner registration with phone OTP, bib numbers with QR codes, sponsor payments through Paystack with PDF invoices and receipts, offline checkpoint scanning, split and finish times, a public tracking page, and a live leaderboard.

Team

Person	Role	Share	Task file
Lead (expert)	Architecture, backend core, scanner, timing	70%	**lead.md**
Frontend 1	Registration and sponsor screens	10%	**frontend-1.md**
Frontend 2	Public race-day pages (leaderboard, tracking, map)	10%	**frontend-2.md**
Backend	Sponsor tiers, pledges, public read endpoints, PDF templates	10%	**backend.md**

Everyone works against the shared contract in **api-contract.md**.

Stack

- **Frontend (`apps/web`):** Next.js (App Router), TypeScript, Tailwind, shadcn/ui, react-hook-form + zod, Leaflet, openapi-fetch.
- **Backend (`apps/api`):** FastAPI, Pydantic v2, SQLAlchemy 2.0, Alembic, ARQ, Redis, PostgreSQL, WeasyPrint + Jinja2, segno, pytest.
- **Services:** Paystack (payments), Termii (SMS OTP), Cloudinary (logos).
- **Local development:** Docker Compose runs Postgres, Redis and the API. Run `pnpm dev` for the web app.

Phases

Phase	Lead	Frontend 1	Frontend 2	Backend
0: Foundation	E1, E2, stub endpoints	Onboarding	Onboarding	Onboarding
1: Registration and sponsors	E3, E4, E5, E6	F1.1–F1.4	F2.1–F2.3 (on mock data)	B1, B2, B4
2: Timing and tracking	E7–E11	Polish and bug fixes	Switch to the real API	B3
3: Hardening	E12, race rehearsal	Rehearsal testing	Rehearsal testing	Rehearsal testing

Onboarding for each newbie:

1. Clone the repo and run it locally.
2. Make a first small PR to learn the Git workflow.
3. Finish the tutorial listed in your task file.

Team rules

1. **One ticket, one branch, one PR.** Name branches `feat/F1.1-registration-form`, `fix/B3-search-empty`, and so on.
2. **Every PR is reviewed by Lead** before merge. Nobody pushes directly to `main`.

3. **Do not touch** the database schema or migrations, auth or OTP, payments or webhooks, the scanner, or the timing engine. If you need a change there, ask Lead.
4. **When you're stuck for more than 2 hours**, post in the team chat with what you tried and the error.
5. **Definition of done:**
   - CI passes (lint, type-check, tests).
   - It works at 375px mobile width.
   - Loading, empty and error states exist.
   - The ticket's acceptance criteria are checked off in the PR description.

Lead (Expert): 70%

You own the architecture and everything risky: data, auth, money, the scanner, and timing. You also unblock and review the three newbies.

First priority: finish E1, E2 and the stub endpoints. Until they exist, the newbies are blocked.

Phase 0: Foundation

E1: Monorepo and tooling (6 pts)

- pnpm workspace containing:
  - `apps/web`: Next.js, Tailwind, shadcn/ui, ESLint, Prettier.
  - `apps/api`: FastAPI with uv, ruff, mypy, pytest.
- `docker-compose.yml` running Postgres 16, Redis 7 and the API with hot reload.
- `pnpm gen:api`: openapi-typescript generates types from `/openapi.json`, wrapped in an openapi-fetch client in `apps/web/lib/api.ts`.
- GitHub Actions: lint, type-check and tests for both apps on every PR. Branch protection on `main`.
- `.env.example` for both apps, and a README with setup steps a newbie can follow on Windows.

E2: Schema and migrations (6 pts)

- SQLAlchemy 2.0 models and Alembic migrations for: `categories`, `runners`, `otp_codes`, `sponsor_tiers`, `sponsors`, `pledges`, `payments`, `checkpoints`, `scans`, `route`.
- Constraints:
  - Unique `runners.phone`.
  - Unique `runners.bib_number`.
  - Unique `scans.client_id`.
  - Unique `(runner_id, checkpoint_id)` on scans; when a runner is scanned twice, keep the earliest scan.
- Seed script with categories, checkpoints and the route GeoJSON.
- **Stub endpoints:** every route in `docs/api-contract.md` returns realistic fake data, so frontend work can start.
- An example CRUD module (`app/modules/categories/`: router, schemas, service, tests) for Backend to copy.

Phase 1: Registration and sponsors

E3: Phone OTP (6 pts)

- Termii integration, with a fake sender in development that logs the code to the console.
- 6-digit code, 10-minute expiry, at most 5 verify attempts.
- Redis rate limits: 1 request per 60 s per phone and 10 per hour per IP.
- Issue a signed `verification_token` (JWT, 30 min) that `/registrations` requires.

E4: Registration and bibs (6 pts)

- Lock the category row (`SELECT … FOR UPDATE`) so caps can't be exceeded when many people sign up at once.
- Assign bib numbers per category from a sequence, e.g. Full `F0001`, Half `H0001`, 10 km `T0001`, Fun run `R0001`.
- The QR code contains an **HMAC-signed token** (bib + runner id), so a printed QR can't be forged by changing the number.
- QR PNG endpoint generated with segno.

E5: Paystack (8 pts)

- Initialize the transaction and return `authorization_url`.
- Webhook handler:

  - Verify the `x-paystack-signature` HMAC-SHA512.
  - Re-verify the transaction through the Paystack API.
  - Handle each event only once, keyed on `reference`.
- Pledge state machine: `pending` → `paid` / `failed`.
- The bank transfer channel is enabled for large sponsors.

E6: PDF pipeline (5 pts)

- ARQ worker container.
- Jobs: render the invoice when a pledge is created and the receipt when it is paid, using Jinja2 + WeasyPrint.
- Upload the PDFs to storage and save their URLs on the pledge.
- Backend writes the templates (B4). You provide the render function and the template context.

Phase 2: Timing and tracking

E7: Offline-first scanner PWA (10 pts)

- Route `/scanner` in `apps/web`.
- A volunteer logs in with a checkpoint PIN, which binds the device to a checkpoint.
- Camera scanning with `html5-qrcode`. Each scan gives a beep and a vibration, and shows the runner's name and bib for 2 s.
- Scans queue in IndexedDB (Dexie) with a client UUID. They sync in batches every 10 s when online, and the screen shows a "pending sync" counter.
- Manual bib entry as a fallback for damaged bibs.
- Must work at full speed in airplane mode.

E8: Scan ingestion (5 pts)

- `POST /scans/batch`:
  - Accepts the device token.
  - Verifies each QR signature.
  - Inserts with `ON CONFLICT DO NOTHING`.
  - Reports results per scan.
- Clock drift: store both device time and server receive time, and flag devices whose clocks are off by more than 2 minutes.

E9: Timing engine (7 pts)

- Elapsed time per scan, measured from the category's gun start.
- Finish time is the scan at the finish checkpoint.
- Overall, category and gender rankings, cached in Redis and recalculated on each scan batch.
- `GET /results` and `GET /runners/{bib}` read their splits and positions from this engine.

E10: Position estimate (4 pts)

- Helper `estimatePosition(route, checkpoints, lastSplit, pace, now)` in `apps/web/lib/tracking.ts` (turf.js), returning `{ lat, lng, distance_km, is_estimate }`.
- Stop the estimate at the next checkpoint so a runner never appears ahead of where they could be.
- Unit tests. Frontend 2 calls this helper; they don't change it.

E11: Live updates (3 pts)

- Redis pub/sub channel `scans`, plus the SSE endpoint `/stream/results` with a heartbeat every 15 s.

Phase 3: Hardening

E12: Ship it (4 pts, plus ongoing reviews)

- Deploy: Vercel (web), Render or Railway (API, worker, Redis), Neon (Postgres). Scale up in race week.
- Locust load test: 5,000 spectators on the leaderboard and SSE, plus 20 scanners syncing.
- Race-day runbook: who to call, how to restart, the manual results fallback.
- Full rehearsal on a test route with volunteers.

Mentoring checklist

- Write every newbie ticket as a GitHub issue with its acceptance criteria.
- Pair for 30 minutes with each newbie on their first ticket.
- Review PRs within 24 hours. Explain the *why* in review comments.

Frontend 1: Registration and Sponsor Screens (10%)

You build what runners and sponsors see when they sign up or pay. Runners will mostly use cheap Android phones on slow data, so design for a **375px screen first** and keep pages light.

Before you start

- Learn the basics:
  - The Next.js Learn course, chapters on App Router, pages and forms.
  - react-hook-form + zod: how to validate a form.
- Run the project locally (see the root README) and open http://localhost:3000.
- Read `docs/api-contract.md`, sections *Auth and registration*, *Categories* and *Sponsors*.
- Call the API **only** through `apps/web/lib/api.ts`, which is generated and typed. Never write `fetch` URLs by hand.

Where your code lives

```
apps/web/app/register/            # F1.1, F1.2, F1.3
apps/web/app/sponsors/            # F1.4
apps/web/components/register/     # your form components
apps/web/components/sponsors/
```

Don't edit `lib/api.ts`, `app/scanner/` or anything in `apps/api/`.

F1.1: Registration form (4 pts)

A multi-step form at `/register`:

1. **Your details:** first name, last name, gender, date of birth, email (optional), phone (Nigerian format; accept `080…` and convert it to `+23480…`).
2. **Category:** cards loaded from `GET /categories`, showing name, distance and "X slots left". Disable full categories and label them "Full".
3. **Emergency contact:** name and phone.
4. **Review:** a summary with Edit buttons, then Continue, which goes to phone verification (F1.2).

Acceptance criteria

- □ Zod validation on every field, with clear error messages under each field.
- □ Runner must be at least 18 for the Full and Half marathon (age on race day). Show the reason if not.
- □ Back and Next keep entered data; refreshing the page doesn't lose it (sessionStorage).
- □ Loading skeleton while categories load, and a retry button if the request fails.
- □ Usable at 375px, with no horizontal scroll.

F1.2: Phone OTP screen (2 pts)

Acceptance criteria

- □ On arrival, call `POST /auth/otp/request` and show "Code sent to +234 80* * 12".
- □ 6-digit input with a numeric keyboard on mobile (`inputMode="numeric"`, `autoComplete="one-time-code"`).
- □ Resend button disabled with a 60 s countdown. Show a friendly message on `429`.
- □ On a valid code, call `POST /auth/otp/verify`, then `POST /registrations` with the form data and `verification_token`.
- □ Handle the `OTP_INVALID`, `OTP_EXPIRED`, `PHONE_ALREADY_REGISTERED` and `CATEGORY_FULL` errors (the last one sends the runner back to step 2).

F1.3: Confirmation page (2 pts)

Acceptance criteria

- □ Shows the name, category, a large **bib number**, and the QR image from `qr_png_url`.
- □ "Download QR" button that saves the PNG, and a print-friendly layout (`@media print`).
- □ Short instructions: "Bring this QR code to bib collection".
- □ Opening the page directly without registering redirects to `/register`.

F1.4: Sponsor pages (2 pts)

1. `/sponsors`: tier cards from `GET /sponsor-tiers` showing name, price in ₦ (convert from kobo, e.g. `₦5,000,000`) and benefit list, plus a "Custom amount" card.
2. `/sponsors/pledge?tier=<id>`: form with company name, contact name, email, phone and website (optional). The custom amount field appears only when no tier is chosen (minimum ₦10,000).
3. Submit to `POST /sponsors/pledges`, then redirect the browser to `authorization_url` (Paystack).
4. `/sponsors/thank-you?reference=…`: poll `GET /sponsors/pledges/{reference}` every 3 s, for at most 1 min, until the status is no longer `pending`. Show:
   - **Paid:** a thank-you message and a "Download receipt" link.
   - **Failed:** a "Try again" link.
   - **Still pending:** "We'll email your receipt."

Acceptance criteria

- □ Money is always shown with `Intl.NumberFormat('en-NG', { style: 'currency', currency: 'NGN' })`.
- □ Never mark anything as paid on the frontend. Only show what the API returns.
- □ The "Download invoice" link (`invoice_pdf_url`) is visible before payment, for sponsors who pay by bank transfer.

Ask Lead when…

- An API response doesn't match the contract.
- You need a new field or endpoint.
- You're stuck for more than 2 hours.

Frontend 2: Public Race-Day Pages (10%)

You build what families and spectators watch during the race: the leaderboard, the runner search, and the live map. On race day thousands of people may open these pages at once on mobile data, so keep them **fast and light**.

Before you start

- Learn the basics:
  - The Next.js Learn course, chapters on App Router and data fetching.
  - React Leaflet basics: a map, a line and markers.
  - MDN: Using server-sent events.
- Run the project locally and read `docs/api-contract.md`, sections *Categories and runners* and *Race day*.
- In Phase 1 the race-day endpoints return **fake data**. Build against it and you'll be ready when the real data arrives in Phase 2.

Where your code lives

```
apps/web/app/results/                # F2.1
apps/web/app/track/                  # F2.2, F2.3
apps/web/components/results/
apps/web/components/track/
```

Don't edit `lib/api.ts`, `lib/tracking.ts` (Lead's helper, you only *call* it), `app/scanner/`, or anything in `apps/api/`.

F2.1: Live leaderboard (4 pts)

Page `/results`.

Acceptance criteria

- ☐ Table from `GET /results`: position, bib, name, category, last checkpoint, and time (`h:mm:ss`). Show the finish time once the runner has finished.
- ☐ Filters for category (tabs) and gender (All, M, F), stored in the URL query (`?category=HALF&gender=F`) so the link can be shared.
- ☐ Pagination or "Load more" (50 per page).
- ☐ Live updates: subscribe to `/stream/results` with `EventSource`. On each `scan` event, refetch the current page at most once every 5 s (throttle it, don't refetch on every event).
- ☐ A "Live" indicator dot that turns grey when the connection drops, with auto-reconnect.
- ☐ On mobile, the table becomes stacked cards (no horizontal scroll at 375px).
- ☐ Empty state: "Results will appear once the race starts".

F2.2: Runner search and detail (3 pts)

Page `/track` (search) and page `/track/[bib]` (detail).

Acceptance criteria

- ☐ Search box calling `GET /runners/search?q=` with a 300 ms debounce and at least 2 characters. Each result links to `/track/{bib}`.
- ☐ Detail page shows name, bib, category, status badge (Registered, Running, Finished, DNF), finish time and positions once finished.
- ☐ Splits table: checkpoint, distance, time of day, elapsed time, and **pace per km** (calculate it yourself from distance and elapsed time).
- ☐ The detail page updates on SSE events for **this bib only**.
- ☐ "Share" button using `navigator.share` when available, otherwise copy the link.

- [ ] A 404 page for an unknown bib.

F2.3: Route map (3 pts)

Add the map to `/track/[bib]`, above the splits table.

Acceptance criteria

- [ ] Leaflet map with OpenStreetMap tiles. Load it with `dynamic(..., { ssr: false })`, because Leaflet needs the browser.
- [ ] Draw the route line and the checkpoint markers from `GET /route`. Markers for checkpoints the runner has passed are green, the others grey.
- [ ] Runner marker at the position returned by Lead's `estimatePosition(...)` from `lib/tracking.ts`, recalculated every 10 s.
- [ ] If `is_estimate` is true, show a pulsing marker with the label "Estimated position". At a checkpoint, show a solid marker.
- [ ] Finished runners show at the finish line. Runners who haven't started show at the start line.
- [ ] Map height 300px on mobile, 450px on desktop.

Ask Lead when...

- The fake data doesn't cover a case you need (for example a DNF runner).
- The SSE or map behaves differently from the contract.
- You're stuck for more than 2 hours.

Backend: Sponsors and Public Read Endpoints (10%)

You build the FastAPI endpoints for sponsor tiers and pledges, the public runner lookups, and the PDF templates. Lead has already set up the database, migrations and an **example module** (`app/modules/categories/`). Copy its structure for every module you build.

Before you start

- Learn the basics:
  - The FastAPI tutorial, from "First Steps" through "SQL Databases" and "Testing".
  - The SQLAlchemy 2.0 ORM quickstart.
- Run `docker compose up`, then open http://localhost:8000/docs (Swagger) and try the example endpoints.
- Read `docs/api-contract.md`, sections *Categories and runners* and *Sponsors*. Your responses must match it **exactly**.

Where your code lives

```
apps/api/app/modules/sponsors/     # B1, B2
apps/api/app/modules/runners/      # B3 (read-only endpoints)
apps/api/app/templates/pdf/        # B4
apps/api/tests/                    # B4 tests
```

Module layout (copied from `categories/`): `router.py` (endpoints), `schemas.py` (Pydantic), `service.py` (database logic), and `tests/`.

Do not touch:

- `app/models/` and `alembic/`: ask Lead if you need a column.
- `app/modules/auth/`, `app/modules/payments/`, `app/modules/scans/` and `app/modules/timing/`.

B1: Sponsor tiers endpoints (3 pts)

Acceptance criteria

- ☐ `GET /sponsor-tiers`: public, ordered by `sort_order`, only active tiers.
- ☐ `POST`, `PATCH` and `DELETE /admin/sponsor-tiers`: protected with Lead's `require_admin` dependency.
- ☐ Validation: name 2–60 characters, `amount_kobo` greater than 0, `benefits` a list of 1–15 non-empty strings.
- ☐ DELETE is a **soft delete** (`is_active = false`), because old pledges still point to the tier.
- ☐ Seed script with Platinum (₦5,000,000), Gold (₦2,000,000) and Silver (₦500,000), each with sample benefits.

B2: Sponsor pledge endpoint (2 pts)

Acceptance criteria

- ☐ `POST /sponsors/pledges` validates that **exactly one** of `tier_id` or `custom_amount_kobo` is sent (custom amount at least 1,000,000 kobo, i.e. ₦10,000), plus the email format and a Nigerian phone number.
- ☐ Creates the `sponsor` and `pledge` rows (status `pending`), then calls Lead's `payments.service.start_checkout(pledge)` and returns its `reference` and `authorization_url`.
- ☐ Enqueues the invoice job with `await enqueue("render_invoice", pledge_id)`.
- ☐ `GET /sponsors/pledges/{reference}` returns the status and receipt URL, and `404` for an unknown reference.
- ☐ You never set the status to `paid`. Only Lead's webhook does that.

B3: Public runner endpoints (3 pts)

Acceptance criteria

- ☐ `GET /categories` adds `slots_left = cap - registered_count` and `is_full`, using one query (no query per category).
- ☐ `GET /runners/search?q=`:
  - If `q` looks like a bib (e.g. `H0123`), match the bib exactly.
  - Otherwise, match the name case-insensitively (`ILIKE`).
  - At most 20 results, and `422` if `q` is shorter than 2 characters.
- ☐ `GET /runners/{bib_number}` returns the shape in the contract, with splits ordered by checkpoint distance. Get splits and positions from `timing.service.get_runner_result(runner_id)`; don't calculate them yourself.
- ☐ **Never** return phone, email, date of birth or emergency contact on these public endpoints. Add a test that checks this.

B4: PDF templates and tests (2 pts)

Acceptance criteria

- ☐ `templates/pdf/invoice.html` and `receipt.html` (Jinja2 + CSS for WeasyPrint). Include:
  - The race logo.
  - Invoice or receipt number.
  - Date.
  - Sponsor company and contact.
  - Tier name or "Custom".
  - Amount in ₦.
  - Organiser bank details (invoice only).
  - "PAID" stamp and the Paystack reference (receipt only).
- ☐ Preview them with Lead's `scripts/render_pdf_preview.py` and attach both PDFs to your PR.
- ☐ pytest coverage for B1–B3:
  - Success cases.
  - Validation errors.
  - Admin protection.
  - Soft delete.
  - Search by name and by bib.
  - The private-fields test.

Ask Lead when...

- You need a database column or a new table.
- A Lead service you call (`start_checkout`, `get_runner_result`, `enqueue`) doesn't behave as described.
- You're stuck for more than 2 hours.

API Contract (v1)

This is the shared agreement between frontend and backend.

- **Base URL:** `/api/v1`. All bodies are JSON.
- **Money:** amounts are integers in **kobo** (₦1 = 100 kobo).
- **Times:** ISO 8601 in UTC. Durations are integer seconds.
- **Errors:** every error looks like `{ "detail": "Human-readable message", "code": "MACHINE_CODE" }`.
- **Types:** frontend TypeScript types are generated from FastAPI's `/openapi.json`. Run `pnpm gen:api` and never hand-write them.

In Phase 0, Lead ships **stub versions** of every endpoint below that return fake data, so frontend work can start straight away.

Auth and registration (owned by Lead)

Method	Path	Body	Response
POST	`/auth/otp/request`	`{ phone }` (format `+234…`)	`204`. `429 OTP_RATE_LIMITED` if requested too often
POST	`/auth/otp/verify`	`{ phone, code }`	`{ verification_token }`, valid for 30 min. `400 OTP_INVALID` / `OTP_EXPIRED`
POST	`/registrations`	`{ verification_token, first_name, last_name, gender: "M"\`	"F", date_of_birth, email?, category_id, emergency_contact_name, emergency_contact_phone }`

Categories and runners (owned by Backend newbie)

Method	Path	Response
GET	`/categories`	`[{ id, code, name, distance_km, cap, slots_left, is_full }]`
GET	`/runners/search?q=`	`[{ bib_number, full_name, category_code }]`, at most 20. `q` is a name or bib, at least 2 characters
GET	`/runners/{bib_number}`	`{ bib_number, full_name, gender, category, status: "registered"\`

Sponsors (tiers and pledges owned by Backend newbie, payments by Lead)

Method	Path	Body	Response
GET	`/sponsor-tiers`		`[{ id, name, amount_kobo, benefits: string[], sort_order }]`
POST	`/admin/sponsor-tiers`	`{ name, amount_kobo, benefits, sort_order }`	`201` tier. Admin only
PATCH	`/admin/sponsor-tiers/{id}`	partial	tier. Admin only
DELETE	`/admin/sponsor-tiers/{id}`		`204`. Admin only
POST	`/sponsors/pledges`	`{ tier_id? , custom_amount_kobo?, company_name, contact_name, email, phone, website? }`. Send exactly one of `tier_id` or `custom_amount_kobo`	`201 { reference, authorization_url, invoice_pdf_url }`
GET	`/sponsors/pledges/{reference}`		`{ reference, company_name, amount_kobo, status: "pending"\`
POST	`/webhooks/paystack`	Paystack payload	Lead only. This is the **only** place a pledge becomes `paid`

Race day (owned by Lead)

Method	Path	Response
GET	`/route`	`{ route: GeoJSON LineString, checkpoints: [{ id, name, distance_km, lat, lng }] }`
GET	`/results?category=&gender=&limit=50&offset=0`	`{ items: [{ position, bib_number, full_name, gender, category_code, last_checkpoint, elapsed_seconds, finish_seconds? }], total }`
GET	`/stream/results`	Server-Sent Events. `event: scan` with data `{ bib_number, checkpoint_id, elapsed_seconds }`
POST	`/scans/batch`	Scanner only. `{ device_id, checkpoint_id, scans: [{ client_id, qr_token, scanned_at }] }`